

EF Core 8.0 Guided Hands-On Exercises — Answer Key

Lab 2: Setting Up the Database Context for a Retail Store

Scenario	The retail store wants to store product and category data in SQL Server.
Objective	Configure DbContext and connect to SQL Server.

Step 1: Create Models

Define plain C# classes (POCOs) representing the entities. EF Core will map these to tables.

```
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; }
    public List<Product> Products { get; set; }
}

public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    public int CategoryId { get; set; }
    public Category Category { get; set; }
}
```

Explanation: Category.Products is a collection navigation property (one category has many products). Product.CategoryId is the foreign key column; Product.Category is the reference navigation property that lets you access the parent category object directly in code.

Step 2: Create AppDbContext

```
public class AppDbContext : DbContext
{
    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer("Your_Connection_String_Here");
    }
}
```

Explanation: DbContext is the session/bridge between the application and the database. Each DbSet<T> property represents a queryable table. OnConfiguring wires the context to SQL Server via UseSqlServer, passing a connection string, e.g.:

```
"Server=(localdb)\mssqllocaldb;Database=RetailInventoryDb;Trusted_Connection=True;"
```

Step 3: Add Connection String in appsettings.json (optional, for ASP.NET Core)

In ASP.NET Core apps, it's best practice to keep the connection string out of code and store it in configuration instead:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=RetailInventoryDb;Trusted_Connection=True;"
  }
}
```

Then read it and register the context in Program.cs / Startup.cs via dependency injection:

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

Why this matters: Storing the connection string in configuration (rather than hard-coded in OnConfiguring) keeps secrets out of source control and lets different environments (Development/Staging/Production) use different databases without code changes. For a plain console app, hard-coding it in OnConfiguring (as in Step 2) is acceptable for learning purposes.

Key Takeaway

AppDbContext plus the two DbSet properties gives EF Core everything it needs to know about your schema. In Lab 3 you'll turn this model into an actual SQL Server database using migrations.